

06b — CORS (Cross-Origin Resource Sharing)

Key Concepts

- **Same-Origin Policy** — browser blocks JavaScript from calling APIs on a different domain by default
 - **CORS** — mechanism that lets servers explicitly allow cross-origin requests via response headers
 - **Origin** — combination of protocol + domain + port (e.g., <https://myapp.com:443>)
 - **Preflight request** — OPTIONS request browser sends before complex requests to check permissions
 - **Wildcard (*)** — allows all origins; insecure and incompatible with credentials
-

How It Works

Simple request:

Browser JS on evil.com → GET <https://myapp.com/api>

Server responds with: Access-Control-Allow-Origin: <https://myapp.com>

Browser checks: is evil.com in the allowed list? No → BLOCKED

Preflight (complex request):

Browser → OPTIONS /api/data ("can I POST with Authorization header?")

Server → 200 + CORS headers ("yes, myapp.com is allowed")

Browser → POST /api/data (actual request sent)

Key CORS Headers

Header	Purpose
Access-Control-Allow-Origin	Which origins are allowed
Access-Control-Allow-Methods	Which HTTP methods are allowed
Access-Control-Allow-Headers	Which request headers are allowed
Access-Control-Allow-Credentials	Whether cookies/auth headers are allowed

Rules

- CORS is enforced by the **browser only** — Postman and curl ignore it completely
 - [Access-Control-Allow-Origin: *](#) cannot be combined with [credentials: true](#) — browser blocks it
 - Always whitelist specific trusted origins in production — never use [*](#) for authenticated APIs
-

Industry Examples

- **Stripe** — explicit origin whitelist; random sites can't call payment API from JS

- **Any SPA + separate API** — React on `app.com` calling `api.com` requires CORS configured on backend
-

Interview Questions

Q: What is CORS and why does it exist?

CORS is a browser security mechanism built on the Same-Origin Policy. Browsers block JS from calling APIs on different domains by default. Servers opt in by including `Access-Control-Allow-Origin` headers listing permitted origins.

Q: What is a preflight request?

An automatic OPTIONS request the browser sends before complex requests (e.g., POST with Authorization header) to ask if the cross-origin request is permitted. The actual request only proceeds if the server responds with correct CORS headers.

Q: Why is `Access-Control-Allow-Origin: *` dangerous?

It allows any website's JavaScript to call your API. A malicious site could make requests on behalf of a logged-in user, with their cookies sent automatically. Also breaks authenticated requests — browsers refuse credentials with wildcard origins.

Q: Does CORS protect your API from all cross-origin requests?

No — CORS is browser-only. Server-to-server calls, Postman, and curl all bypass it. CORS only controls browser JavaScript behavior.